

Understanding I/O, Event-Driven Systems & Echo Server

By Anvi Bansal

Background: The Server Challenge



A robust server (web, ssh, ftp) must handle multiple clients simultaneously while maintaining:

- Low Performance Impact
- High Throughput
- Responsiveness

Problems with Traditional servers:

- Handles one request at a time
 - Requires multiple threads/processes
 - High memory and CPU usage
 - Poor scalability
- 

What is I/O?

Input- receiving data

Output- sending data

Client → Server → Client

“I/O is how systems communicate by exchanging data.”

In Node.js, I/O often refers to reading/writing files or network operations

Different Types of I/O



Blocking I/O

- Program **waits** until task finishes
- Eg- read() → wait until data arrives

Problem:

- Server gets stuck
 - Cannot handle multiple users efficiently
- 

Different Types of I/O

Blocking I/O

In a blocking model, execution flow halts at the system call.

- Step 1: App calls read().
- Step 2: OS begins fetching data from disk.
- Step 3: Thread is blocked; CPU switches to another process.
- Step 4: Data arrives; OS wakes up the thread.
- Step 5: App resumes execution with data.

Analogy: Waiting in line at a fast-food counter until your specific order is cooked and handed to you.

Different Types of I/O

Non-Blocking I/O

- Program does not wait
- if no data → move on

Result:

- Server stays responsive
 - Can handle many clients
-

Different Types of I/O

Non-Blocking I/O

- In a non-blocking model, the application must frequently check (poll) if the operation is done.
- Step 1: App calls read() (set to non-blocking).
- Step 2: OS says "Not ready yet!" (returns immediately).
- Step 3: App does other work.
- Step 4: App calls read() again later.
- Step 5: OS returns data if ready.

Analogy: Taking a pager at a restaurant. You can sit down and talk until it buzzes, rather than standing at the counter.

Different Types of I/O

Synchronous

- The program waits for the operation to finish before moving on

Asynchronous

- The program starts the operation and continues doing other work and handle the result later using events or callbacks.

Blocking/non-blocking is about whether the program waits, while synchronous/asynchronous is about how the execution flow is structured."

C Server

- C/C++ is used in this task to understand low-level concepts like sockets and blocking I/O

Node.js Server

- JavaScript runtime that uses **non-blocking asynchronous** I/O and an **event loop** to handle multiple clients efficiently.



Event-Driven I/O

The solution to managing non-blocking operations efficiently.



What is the **Event Loop**?

Polling non-blocking sockets continuously wastes CPU cycles. Event-driven I/O solves this using an **Event Loop**.

Instead of the application asking "Are you done yet?", it tells the OS: "Here is a list of tasks. Put my thread to sleep, and wake me up when any of these events happen."

This is the mechanism that allows a single thread to concurrently handle thousands of connections without stalling.

Node.js & The Event Loop

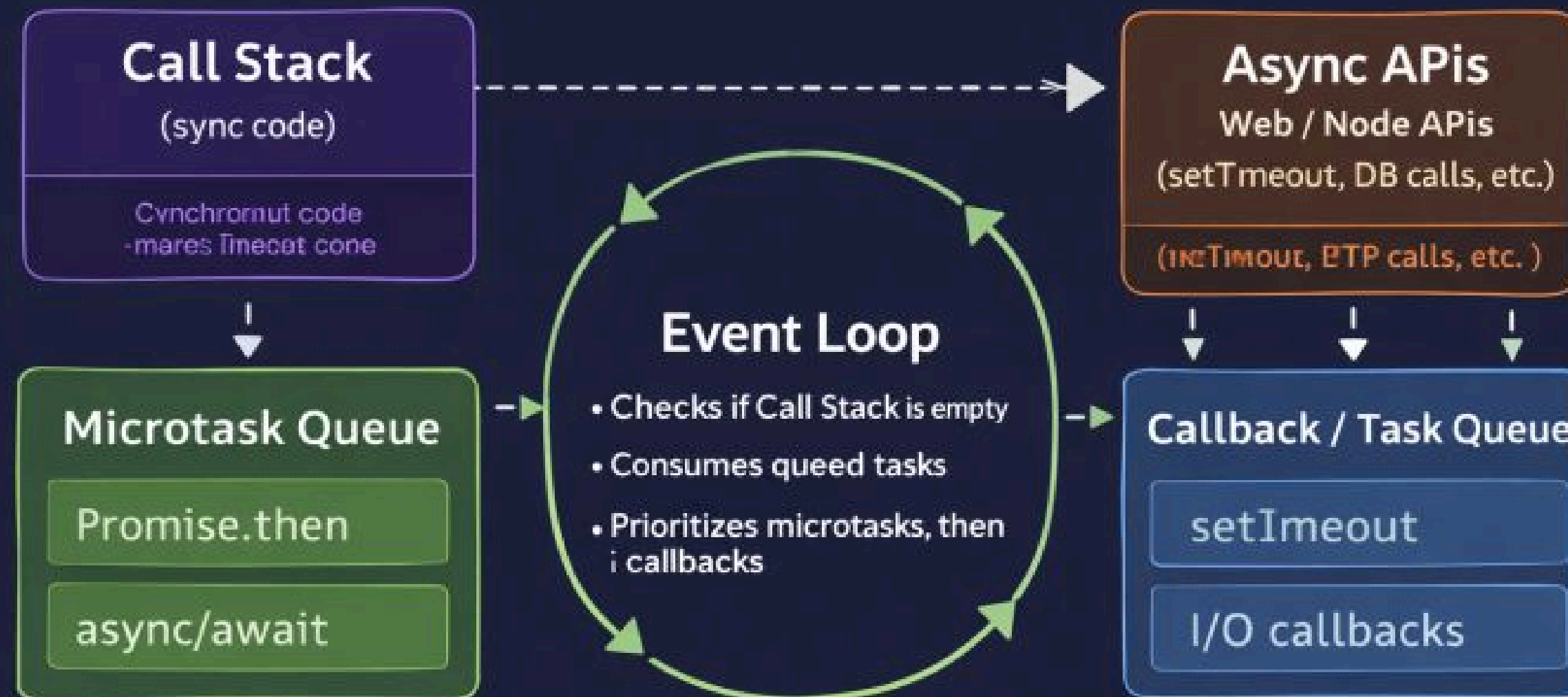
Node.js is built entirely around the concept of non-blocking, event-driven I/O. It uses a library called **libuv** to implement the event loop.

Despite JavaScript being single-threaded, Node.js can perform heavy I/O operations by offloading them to the system kernel whenever possible.

When the kernel finishes the task, it tells Node.js, and the appropriate callback is added to the poll queue to eventually be executed.

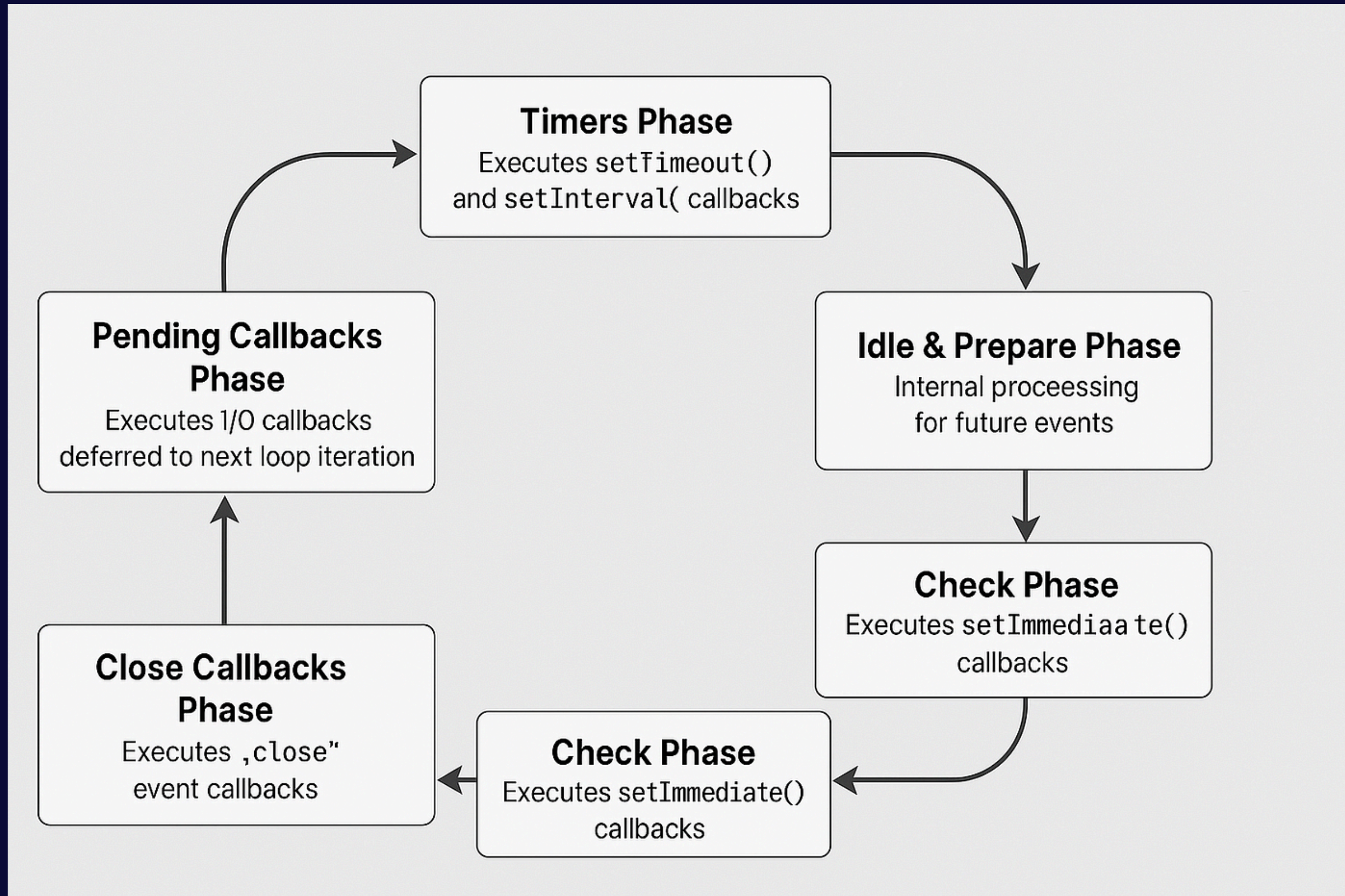
Understanding the Event Loop (Node.js / JavaScript)

How JavaScript handles asynchronous tasks



- **Call Stack** runs sync code
- **Microtask Queue**: Promises, async/await
- **Callback Queue**: Timers, I/O, etc.

Node.js Event Loop Phases



Conclusion

- Blocking I/O → inefficient
- Non-blocking & Async → efficient
- Event-driven model → scalable
- Event loop → handles multiple tasks
- Foundation of modern systems like Node.js

The background is a dark blue gradient. In the four corners, there are abstract geometric patterns. The top-left and bottom-left corners feature a network of light blue lines with circular nodes. The top-right and bottom-right corners feature a pattern of light blue hexagons. Two thin, horizontal light blue lines are positioned above and below the central text.

Thank You